

Retrieval + VLM Gate — Setup and Run Guide

Nishank Satish

September 1, 2026

This is a step-by-step guide to getting the retrieval half of the project running from a fresh clone. Follow the steps in order — each one depends on the one before it. Nothing here needs you to understand embeddings or retrieval; for what the system actually does, see `RETRIEVAL_README.md` in the project root instead.

1 What you need before starting

- This repository, on the correct branch.
- An NVIDIA API key (free) for the VLM step.
- The `data` folder — sent separately on the shared Google Drive, not in git.
- Python 3.10 or later, and `pip`.

2 Step 1 — Get the code

Clone the repository and switch to the branch with the retrieval work on it. `main` and `feat/object-detection-YOLO` do not have any of this — it is all on its own branch:

```
git clone \
    git@github.com:terriljoel/retrieval-grounded-remote-sensing.git
cd retrieval-grounded-remote-sensing
git checkout feat/retrieval-vlm-gate
```

3 Step 2 — Python environment

Two requirements files, one for each half of the project:

```
pip install -r requirements.txt
pip install -r requirements-retrieval.txt
```

4 Step 3 — API key

The VLM step (the second AI model that reads uncertain detections) calls NVIDIA’s hosted API. Get a free key at <https://build.nvidia.com> (any model page, then “Get API Key”), then:

```
cp .env.example .env
```

Open `.env` and fill in:

```
NIM_API_KEY=nvapi-...your key...
NIM_MODEL=nvidia/nemotron-3-nano-omni-30b-a3b-reasoning
```

The `NIM_MODEL` line matters — the code’s built-in default model was retired by NVIDIA and will fail with an error if you skip this.

`.env` is never read directly by the code and is never committed to git. Before running anything that calls the VLM, load it into your shell:

```
export NIM_API_KEY='nvapi-...'
export NIM_MODEL='nvidia/nemotron-3-nano-omni-30b-a3b-reasoning'
```

or, equivalently, `set -a; . ./.env; set +a`.

5 Step 4 — Place the dataset

The NWPU VHR-10 dataset (satellite images with labelled objects) is the only raw dataset anything in this guide needs. It goes at:

```
docs/shared_resources/datasets/raw/NWPU VHR-10 dataset/
  positive image set/
  negative image set/
  ground truth/
```

These exact folder names matter — the code reads this layout directly. Ask for this dataset if you were not sent it separately; nothing in the project runs without it.

(HRRSD is a second dataset that shows up elsewhere in this repository, but nothing in this guide needs it — it is used by one optional script that already ran and produced its output, and neither the demo app nor any of the measurement scripts in Step 6 touch it. Skip it.)

The data folder you receive on the shared Drive replaces (or merges into) the project’s own `data/` directory at the repository root:

```
data/
  memory/      <- verified-case database. Required for the demo app.
  crops/       <- cached crop images. Optional.
  embeddings/  <- cached embeddings. Optional.
```

Only `data/memory/` is strictly required (for the demo app in Step 7). The others are caches — every script rebuilds them automatically if they are missing, just more slowly on first run.

6 Step 5 — Model weights

Nothing to do here. The RemoteCLIP encoder (about 2.3 GB) downloads itself automatically from Hugging Face the first time any script runs, as long as you have an internet connection. It is cached afterwards under `checkpoints/` and will not re-download.

7 Step 6 — Run the measurement scripts

These reproduce every number in `reports/FINDINGS.md`. Each one is self-contained — run from the repository root:

```
python3 -m scripts.run_gate_test      # does retrieval work at all?
python3 -m scripts.run_baselines      # the two simple baselines
python3 -m scripts.run_alpha_sweep    # tuning the crop fusion weight
python3 -m scripts.run_open_set       # spotting an unseen class
python3 -m scripts.run_label_noise    # robustness to bad labels
python3 -m scripts.run_arm_ablation   # which crop views help
python3 -m scripts.run_conformal      # calibrating the accept threshold
python3 -m scripts.run_b3_vlm \
    --scoring logprob --evidence none
```

Each writes its results to `reports/gate/` or `reports/phase1/` and prints a summary to the terminal. The first run of each is slower (building crops and embeddings); later runs read from cache and are quick.

8 Step 7 — The real detector export (optional, but needed for the demo)

Two things in this project need a real YOLO detection export, not just the dataset: the interactive demo app, and `scripts/run_b0.py` (the real-vs-synthetic comparison). Place the export at:

```
data/detections/detections.csv
data/detections/inference_images.csv  <- if your export has one
```

If you do not have this file, ask for it, or generate your own by running the detector’s own export step (see the main `README.md` for that half of the project).

9 Step 8 — Run the demo app

```
streamlit run app/gate_explorer.py
```

Open the URL it prints. Pick an image, pick a detection, click “Find similar cases,” click “Ask the VLM,” and see the gate’s verdict next to the ground truth. `docs/UI_TEST_CASES.md` has four specific cases worth clicking through, with what each one is supposed to show.

10 If something goes wrong

- **A script says a file is not found under `docs/shared_resources`** — the dataset is not in the right place; recheck Step 4’s folder names exactly.
- **The VLM step raises an error naming `NIM_API_KEY`** — the key is not loaded into your shell; see Step 3.
- **The VLM step raises an HTTP error about a model** — `NIM_MODEL` is not set, or is set to the retired default; see Step 3.
- **The app says `detections.csv` is missing** — see Step 7.

- **Anything else** — the error messages in this codebase are written to say exactly what is missing and where it was expected; read the message before asking, it usually says the fix directly.

11 Where to read more

RETRIEVAL_README.md What this half of the project does, the eight-step pipeline explained in plain English, and the headline results.

reports/FINDINGS.md Every result from this project, numbered, tagged by whether it is measured on real data or synthetic proposals only.

reports/gate/GATE_REPORT.md The first test of whether retrieval works at all.

docs/UI_TEST_CASES.md Four worked examples for the demo app, with what each one is supposed to show.

docs/STATE.md The full running log of decisions and findings, if you want the complete history rather than the summary.